Lab Midterm Examination

Assignment Submission Portal with Automated Testing

Core marks 40 (viva:15) Bonus marks 10

Scenario

Your department needs a simple assignment submission portal. Previous updates have occasionally broken submission functionality, so every code change must be tested automatically. The instructor also needs a local record of the test results. Develop the portal using any programming language and implement the requirements below.

1 Develop the assignment submission portal — 8 marks

Allow students to enter their name, registration number, assignment title, and submission URL. Validate required fields and the URL, save valid submissions, and display the saved submissions. A submission link is sufficient; file uploading is not required.

2 Maintain the project on GitHub — 4 marks

Store the application, automated tests, pipeline configuration, and local report receiver in a GitHub repository. Include a README explaining how to run the application, execute the tests, and start the receiver.

3 Run automated tests on every push — 10 marks

Configure a continuous integration (CI) pipeline that automatically tests the pushed revision after every push to the repository. Implement at least the following three test cases.

Test case	Expected result
Complete details and a valid submission URL	The submission is saved and appears in the submission list.
Missing registration number	The submission is rejected with a validation message.
Invalid submission URL	The submission is rejected and is not saved.

Results must come from executing the tests. A failed test must cause the test stage to fail.

4 Send the report to localhost and save JSON 14 marks

Implement a local HTTP receiver that automatically receives the completed test report and saves it as test-report.json on your computer. Include the following information in the report:

- Repository name, branch, commit SHA, pipeline run identifier, and execution timestamp.
- Each test's name, result, and failure message where applicable.
- Total, passed, and failed test counts, together with the overall test status.

Deliver the report for both passing and failing test runs. Manual downloading or copying does not satisfy this requirement. A GitHub push notification alone is insufficient; the file must contain actual test results.

Connectivity note: A GitHub-hosted runner's localhost refers to the runner. Use a tunnel or relay to reach your computer, or run the CI runner locally. Keep the receiver and any forwarding client running during the demonstration.

5 Demonstrate the complete workflow 4 marks

Demonstrate a push that passes all tests. Introduce a defect and push again to produce a failing test. Show the pipeline results and the automatically updated local JSON file for both runs. Finally, fix the defect and demonstrate a passing run.

Bonus tasks

Conditional deployment - 5 marks. Automatically publish the portal's static frontend or demo to GitHub Pages after a push to main, only when all tests for that revision pass. A failing run must leave the previously published version unchanged. Provide the deployed URL.

GitHub Pages supports static websites and does not execute a server-side backend. A static frontend or demo is sufficient for this bonus.

Email report - 5 marks. Automatically send the test report to a configurable email address after both passing and failing test runs. Include the commit identifier, overall result, test counts, and individual results or an attached report. Keep email credentials in CI secrets or environment variables.

Submission requirements

Submit the GitHub repository URL, setup instructions, and evidence of the passing, failing, and corrected demonstrations. Include the Pages URL and received email evidence if attempting the bonus tasks.